DIGITAL FORENSICS AND STEGANOGRAPHY

Deleted JPG Recovery and Hidden-Data Extraction

COURSE-END PROJECT REPORT

Case ID: DF-2026-001

Environment: Kali Linux in Oracle VirtualBox

Filesystem: NTFS

Forensic image: 2 GB virtual disk image

Prepared from the practical terminal evidence and screenshots supplied.

Abstract

This project demonstrates a controlled digital-forensics workflow involving image steganography and deleted-file recovery. A text file containing simulated confidential laboratory information was embedded in a JPEG image using Steghide. The image was copied to an NTFS-formatted virtual disk and deleted. A forensic acquisition copy was then created and examined with Sleuth Kit. The deleted JPEG was identified and recovered, after which the hidden text was extracted from the recovered image. SHA-256 comparisons were used to verify the integrity of the acquisition and recovered content.

1. Objectives

- Create a sample JPEG and embed a text file using Steghide.
- Create a virtual disk image with an NTFS partition.
- Copy the steganography-enabled image to the NTFS volume and delete it.
- Create a forensic acquisition copy and preserve its integrity using SHA-256.
- Identify the deleted JPG using Sleuth Kit and recover it using icat.
- Extract the hidden text from the recovered JPG.
- Compare hashes to confirm that the recovered files match their originals.

2. Tools and Environment

Tool	Purpose
Kali Linux / Oracle VirtualBox	Controlled forensic lab environment
Steghide 0.6.0	Embedding and extracting hidden data in a JPEG
Sleuth Kit 4.14.0	Partition and deleted-file analysis using mmls, fls, and icat
NTFS-3G / mkfs.ntfs	Creating and mounting the NTFS filesystem
SHA-256 (sha256sum)	Integrity verification
losetup / parted / cp	Virtual disk attachment, partitioning, and acquisition copy
file	Identifying the recovered file format

3. Methodology and Implementation

3.1 Workspace and tool preparation

A project workspace was created at ~/forensics-project with directories for images, evidence, recovered files, reports, and disk images. The supplied screenshots show the installation or verification of Steghide, Sleuth Kit, PhotoRec, ExifTool, NTFS utilities, and supporting tools. The versions displayed include Steghide 0.6.0, Sleuth Kit 4.14.0, and PhotoRec 7.2.

3.2 Preparing the hidden message and JPEG

The text file secret.txt was created for case DF-2026-001. It describes simulated confidential information and states that the experiment demonstrates image steganography and deleted-file recovery. A 1200 × 800 sample JPEG was generated. Steghide embedded secret.txt into images/hidden_evidence.jpg using a passphrase. Steghide later reported an embedded file named secret.txt, with a size of 210 bytes, encrypted using rijndael-128 in CBC mode and compressed.

3.3 Creating the NTFS virtual disk

A 2 GB virtual disk image was created at disk/forensic_disk.img. A DOS/MBR partition table and a primary NTFS partition were created. The image was attached using a loop device, the partition was formatted as NTFS, and it was mounted at /mnt/forensic_ntfs.

3.4 Copying and deleting the evidence image

The steganography-enabled JPG was copied to /mnt/forensic_ntfs/. The source and copied JPG had the same SHA-256 hash. The file command identified the copy as a JPEG image with dimensions 1200 × 800. The JPG was then deleted from the mounted NTFS volume.

3.5 Forensic acquisition and deleted-file analysis

The NTFS volume was unmounted and the loop device detached. The original virtual disk image was hashed and copied to evidence/forensic_acquisition.img. The original and acquisition-copy hashes matched. The mmls command identified a DOS partition table and an NTFS/exFAT partition beginning at sector 2048. The fls command, run recursively with deleted entries enabled, listed hidden_evidence.jpg as a deleted entry with metadata address 64-128-2.

3.6 Recovering the JPG and extracting hidden data

The icat command recovered metadata address 64-128-2 from the acquisition image using partition offset 2048. The output was saved as recovered/recovered_hidden_evidence.jpg. The recovered file was identified as a valid 1200 × 800 JPEG. Steghide confirmed that the recovered image still contained secret.txt. The embedded text was extracted to recovered/final_secret.txt.

4. Results and Integrity Verification

The practical work produced the following SHA-256 comparisons:

Comparison	Result	SHA-256 value
Original JPG vs recovered JPG	MATCH	dd950137c1471ae0f148aadf92c44094ace0ddf0b67ed91d7a73b571670de33d
Original secret.txt vs extracted text	MATCH	1425ece7553a8f97191abc8c941bf1b809213bc562afd52d56439c7d3780c63d
Original disk image vs acquisition copy	MATCH	a0d0cb569f451220584b4c526532c8181950dcb8f5f0ddee6cd87ce61807a8c6

All three comparisons shown above produced matching SHA-256 values. The extracted text was readable and contained the case identifier DF-2026-001 and the stated laboratory objective. These results support that the recovered JPG and extracted text match the corresponding original files, and that the acquisition copy matches the original disk image.

5. Conclusion

The project successfully demonstrated image steganography, NTFS file storage, deletion, forensic acquisition, deleted-file identification, file recovery, and extraction of hidden data. Sleuth Kit identified the deleted JPG in the NTFS partition, and icat recovered the file from the forensic acquisition image. Steghide extracted the embedded text from the recovered JPG. Matching SHA-256 values verified the integrity of the recovered JPG, extracted text, and forensic acquisition copy.

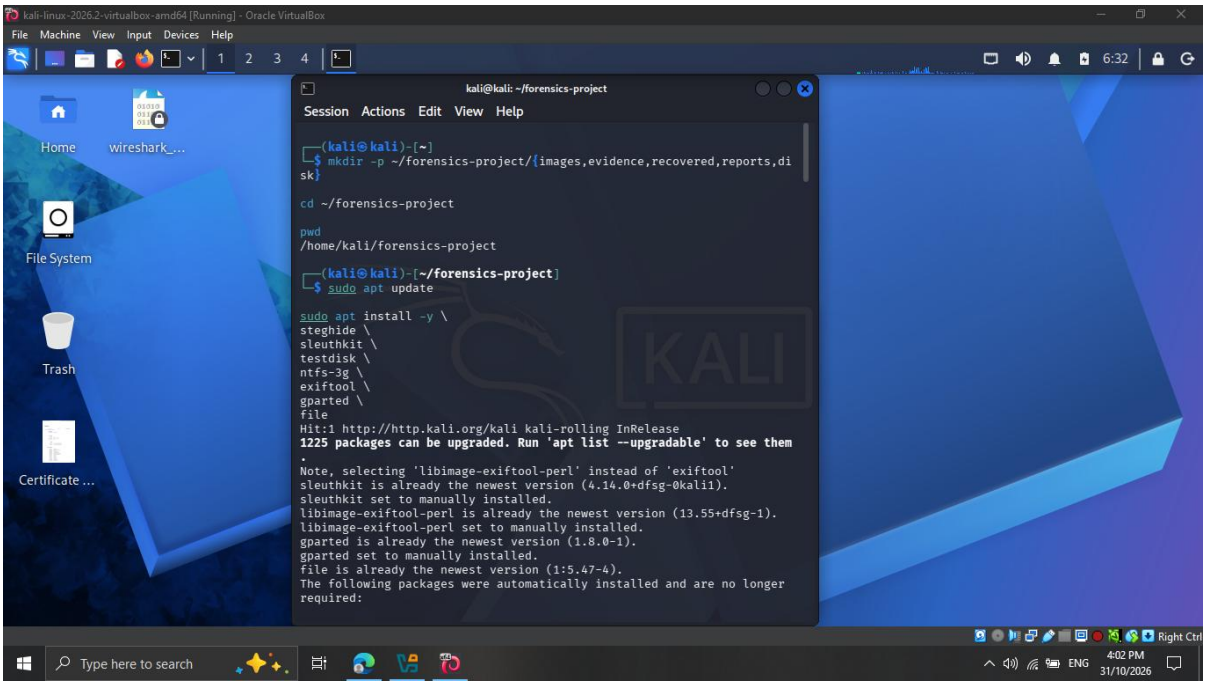
6. Scope and Notes

- This was a controlled laboratory simulation using a virtual disk image.
- The recovery succeeded because the deleted file's data remained recoverable and had not been overwritten.
- The practical screenshots show the use of Sleuth Kit commands mmls, fls, and icat. FTK Imager is not shown as having been used in this implementation.
- The NTFS partition is reported by mmls as beginning at sector 2048, which was used as the filesystem offset during analysis.

Appendix A: Practical Screenshots

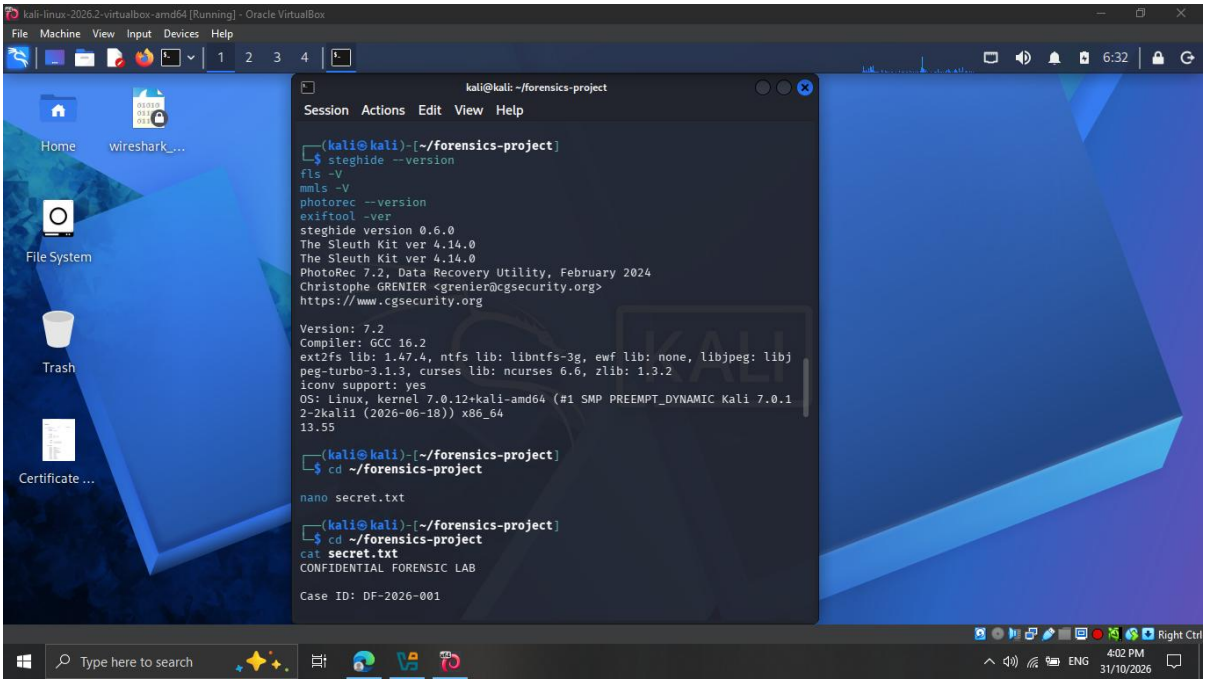
The screenshots below are included in the order supplied and document the project setup, steganography, NTFS disk preparation, deletion, acquisition, recovery, and validation.

A1. Project workspace and initial forensic lab setup



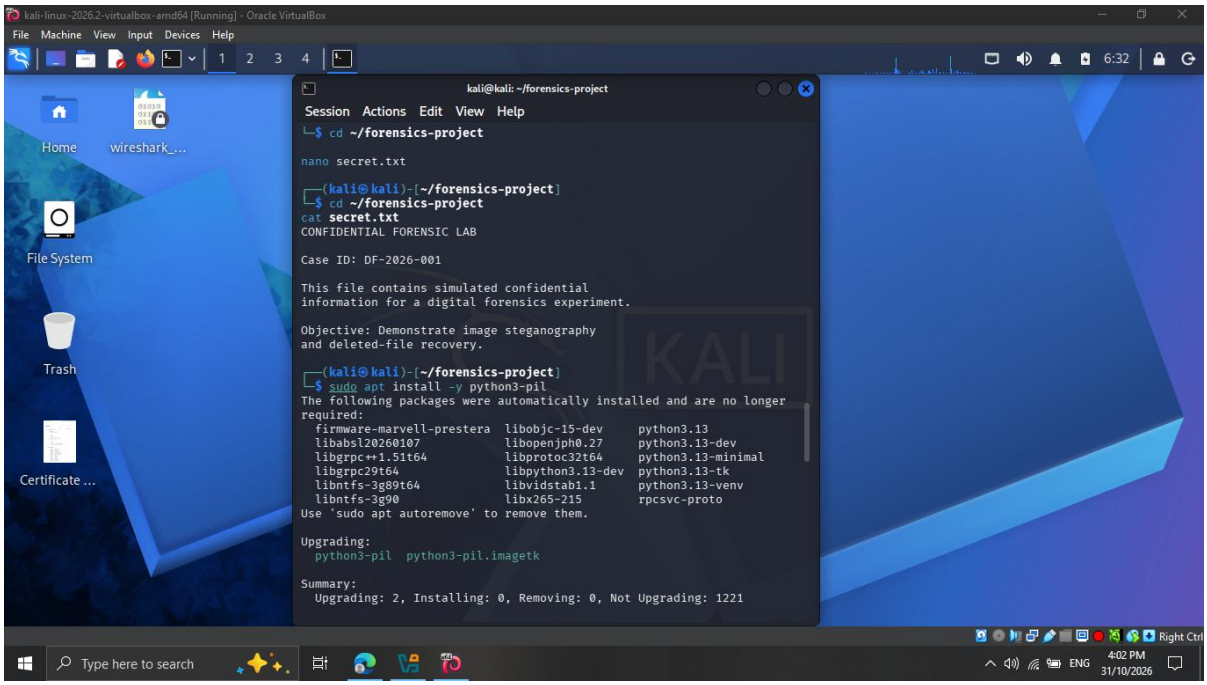
Screenshot (1269).png

A2. Verification of installed forensic tools and creation of the case secret file



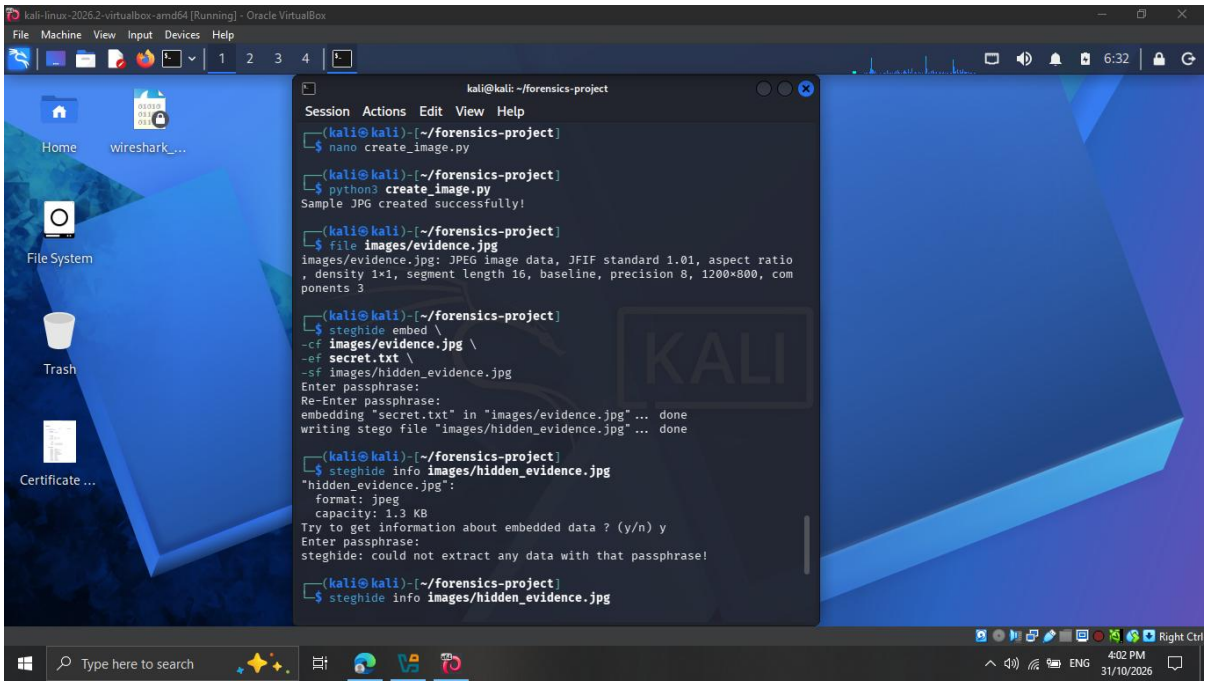
Screenshot (1270).png

A3. Secret-file contents and preparation of the sample image



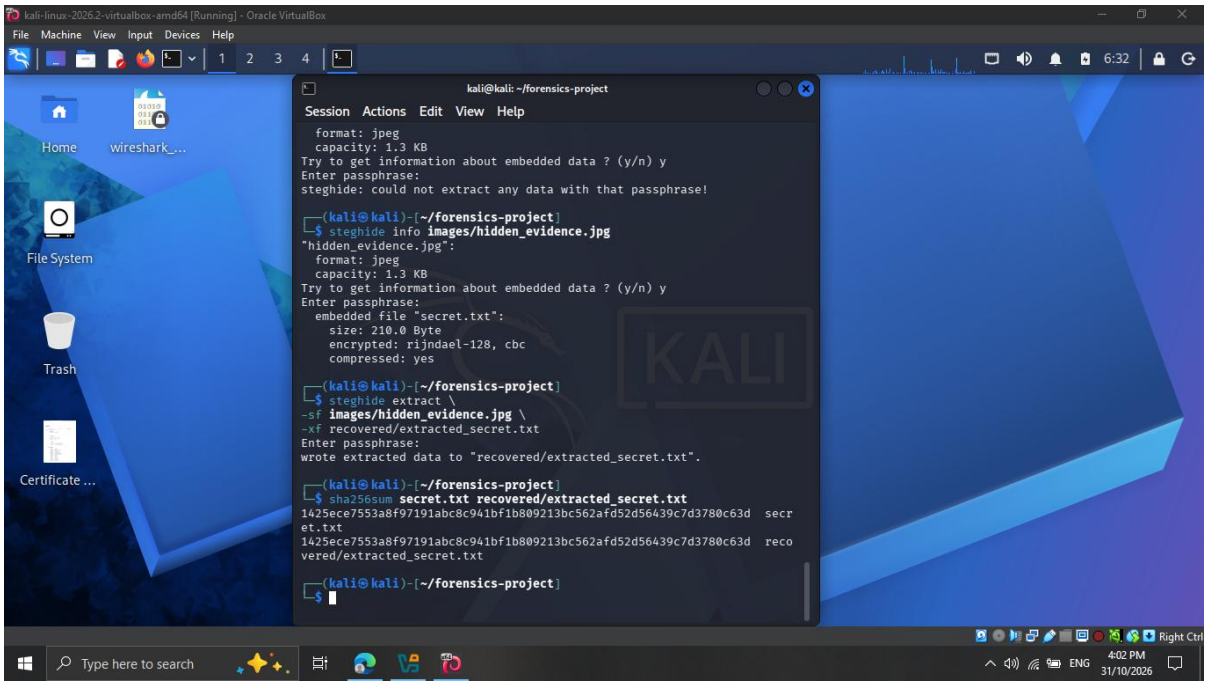
Screenshot (1271).png

A4. Creation of the sample JPEG and embedding the secret with Steghide



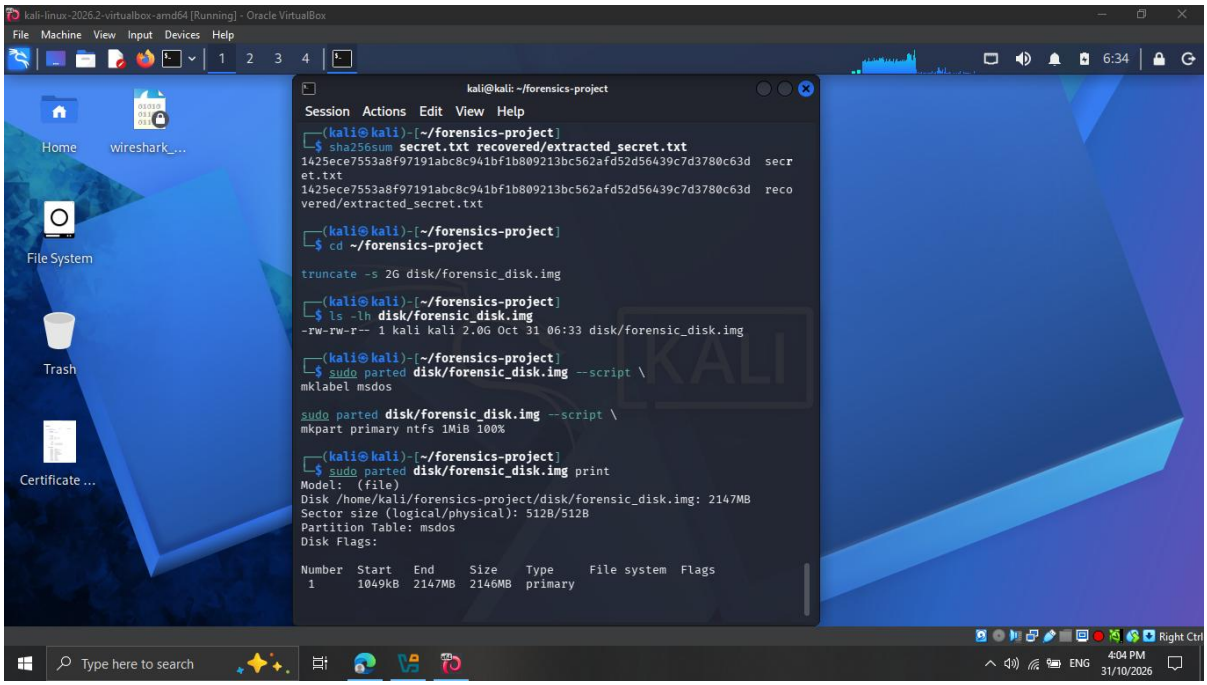
Screenshot (1272).png

A5. Steghide inspection and extraction of the embedded text



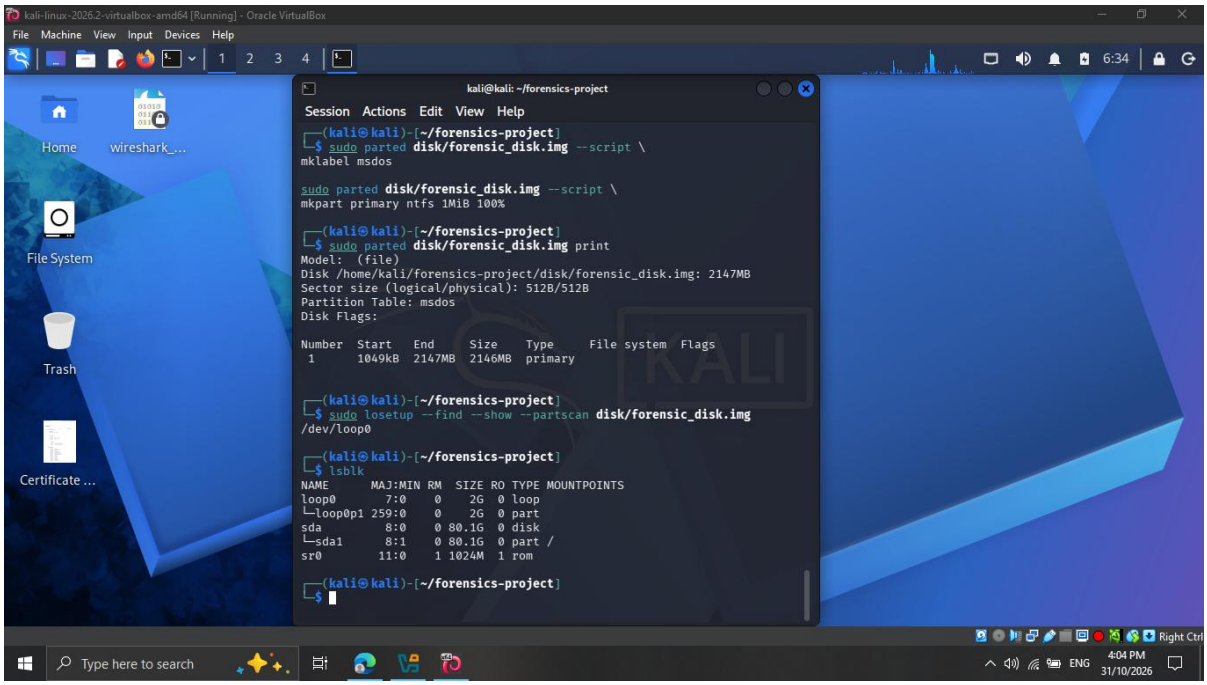
Screenshot (1273).png

A6. Verification that the original and extracted secret files have matching hashes



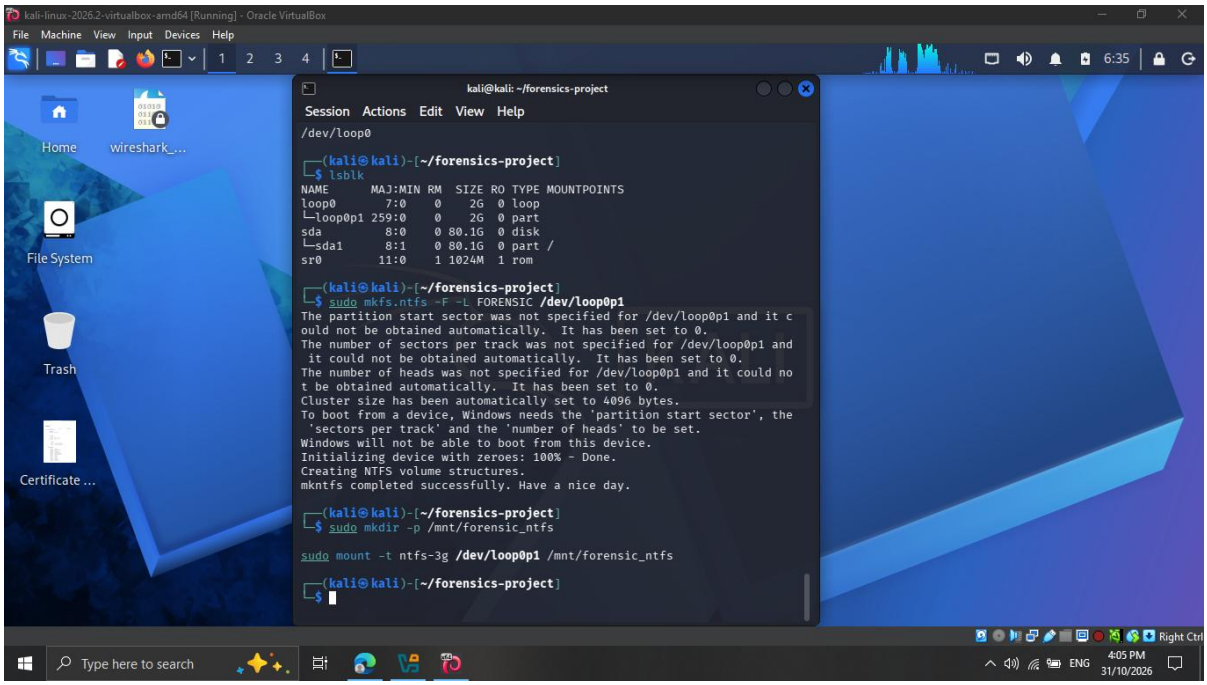
Screenshot (1274).png

A7. Creation and partitioning of the forensic virtual disk image



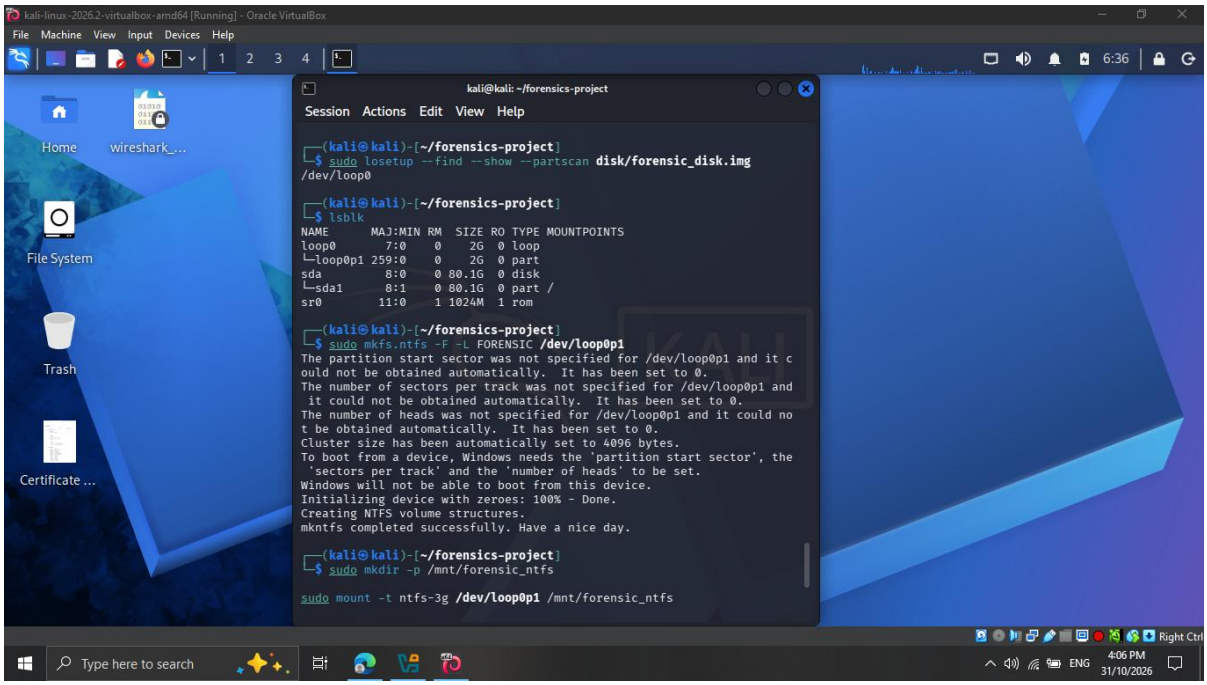
Screenshot (1275).png

A8. Loop-device and NTFS filesystem preparation



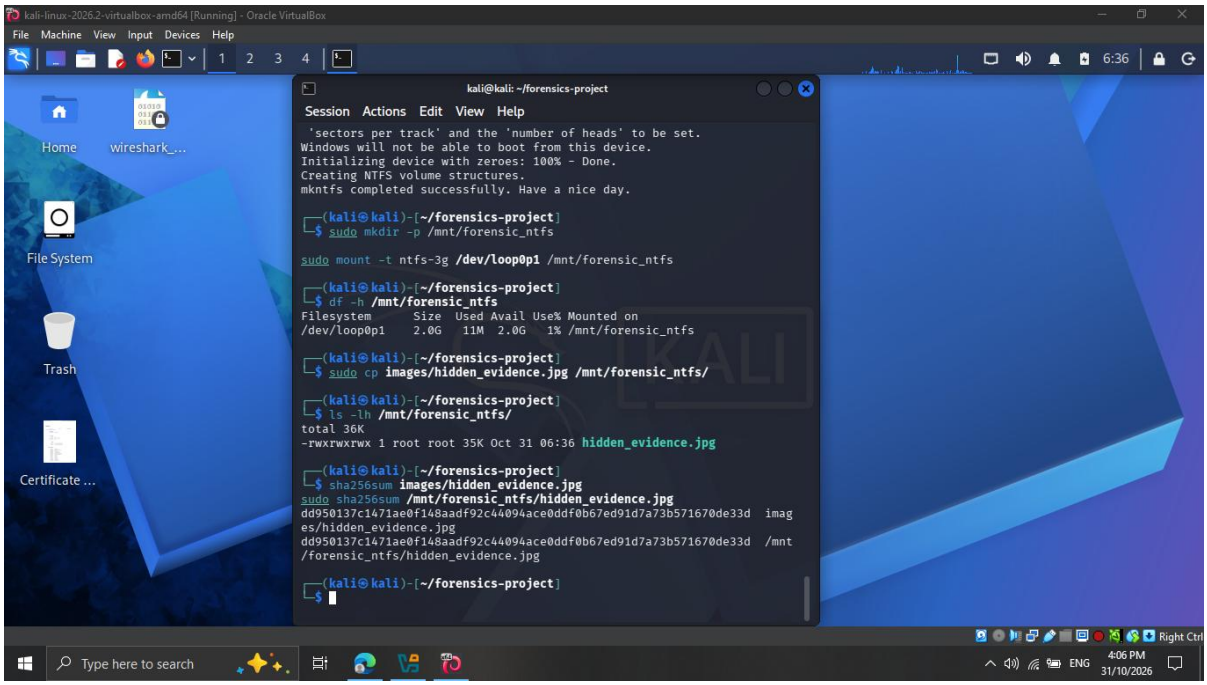
Screenshot (1276).png

A9. Mounting the NTFS partition and checking available filesystem space



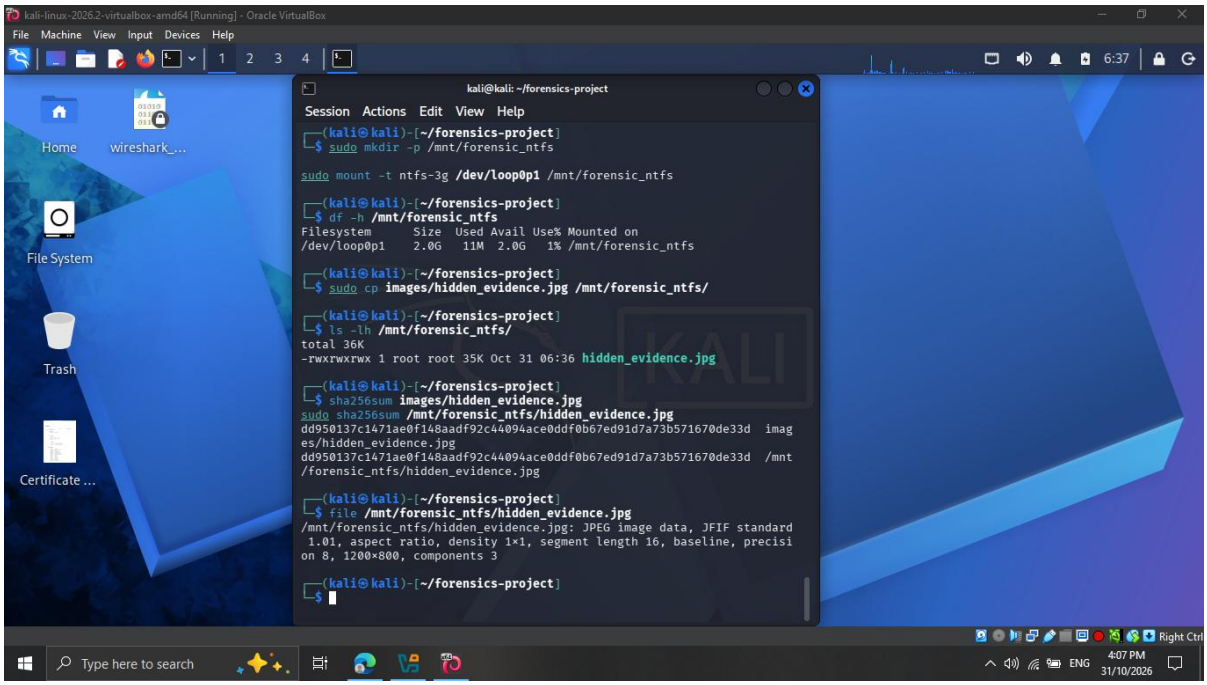
Screenshot (1277).png

A10. Copying the JPG to the NTFS volume and verifying its hash



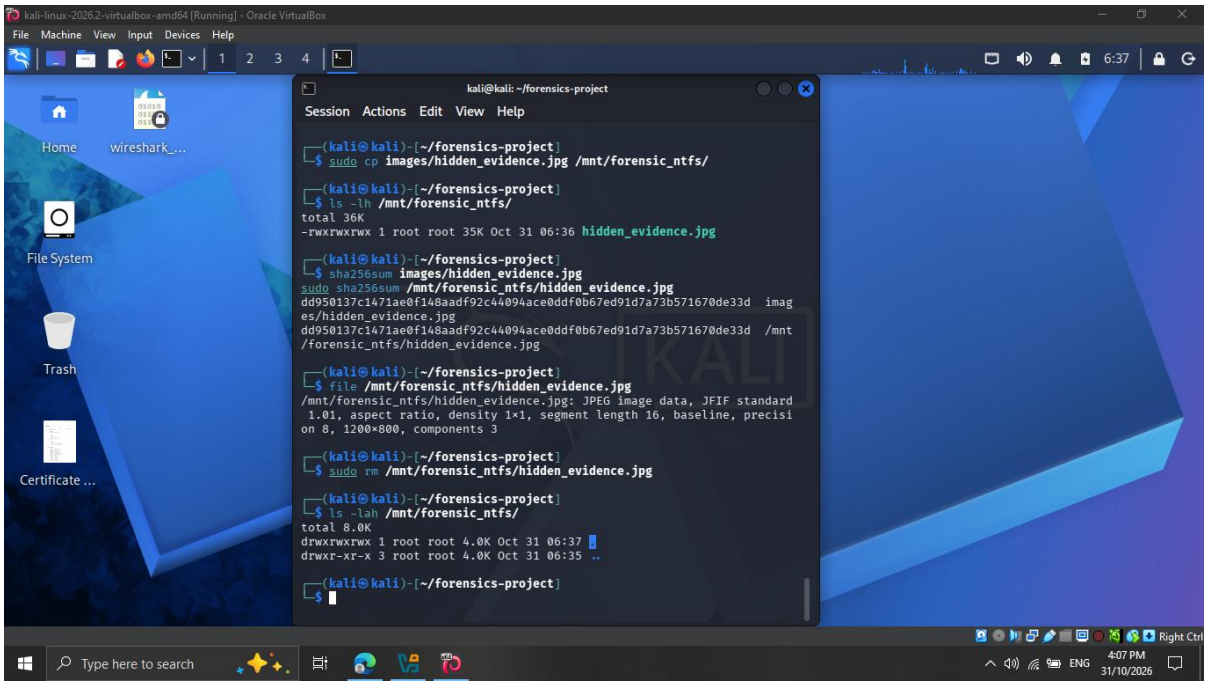
Screenshot (1278).png

A11. Verifying the JPG on the NTFS volume and deleting the file



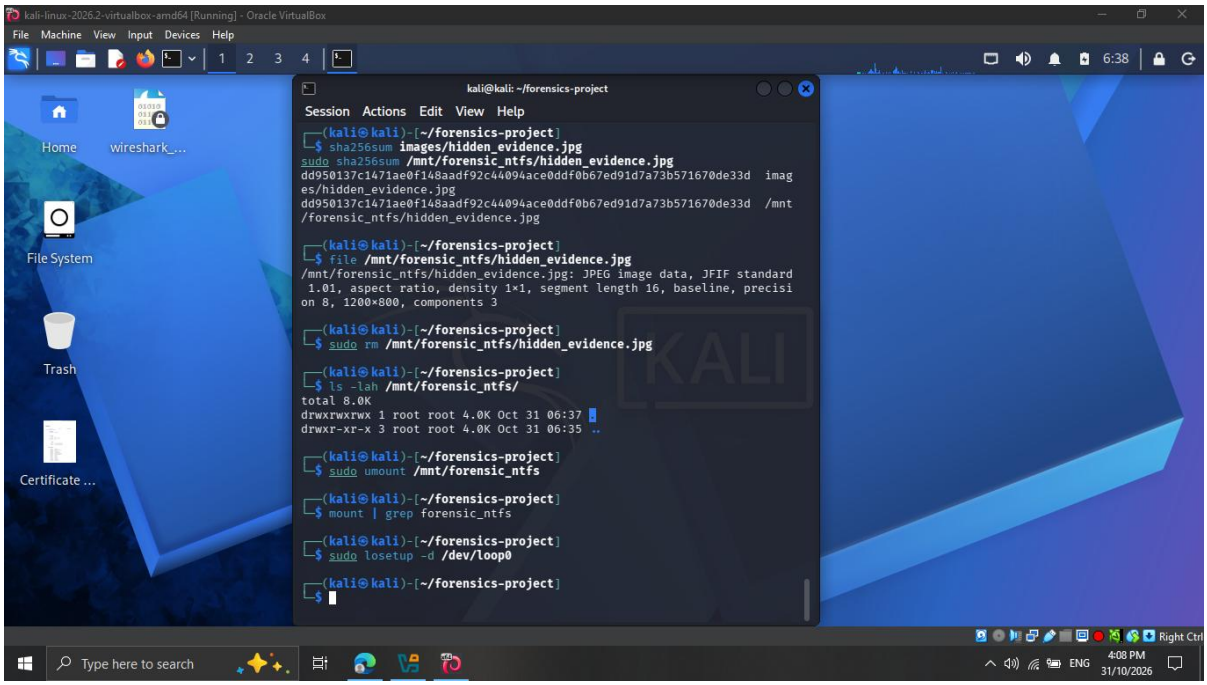
Screenshot (1279)(1).png

A12. Confirming deletion and detaching the forensic volume



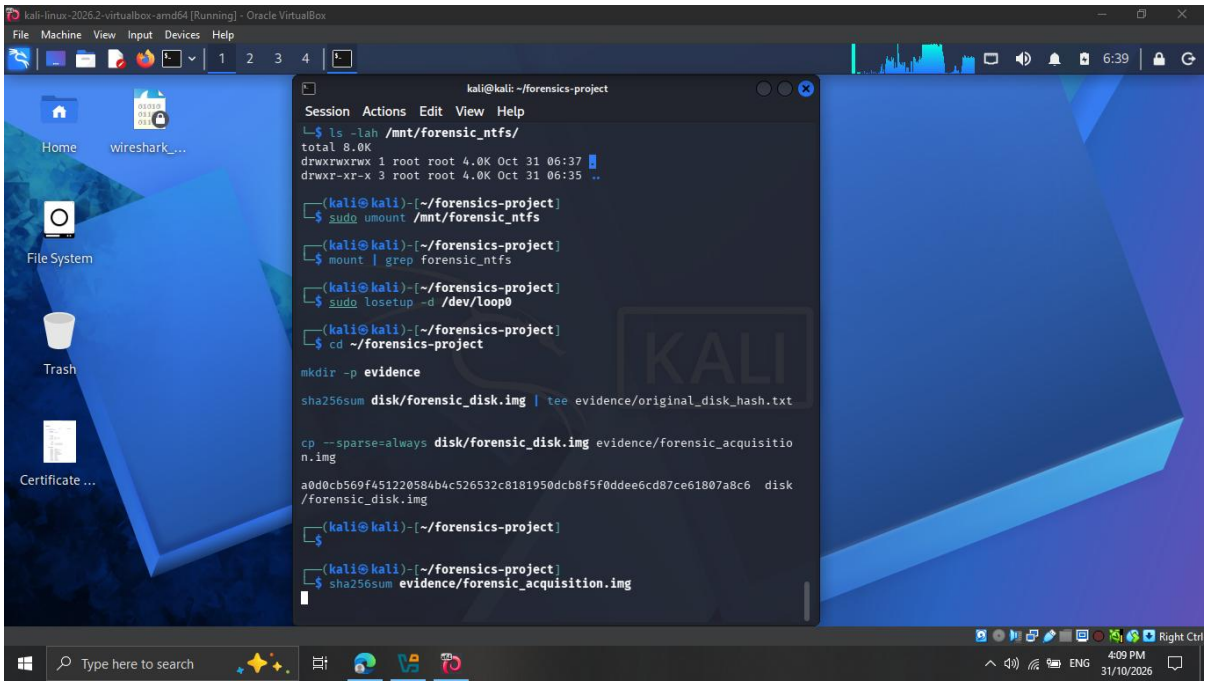
Screenshot (1280)(1).png

A13. Creating the forensic acquisition copy and recording the original image hash



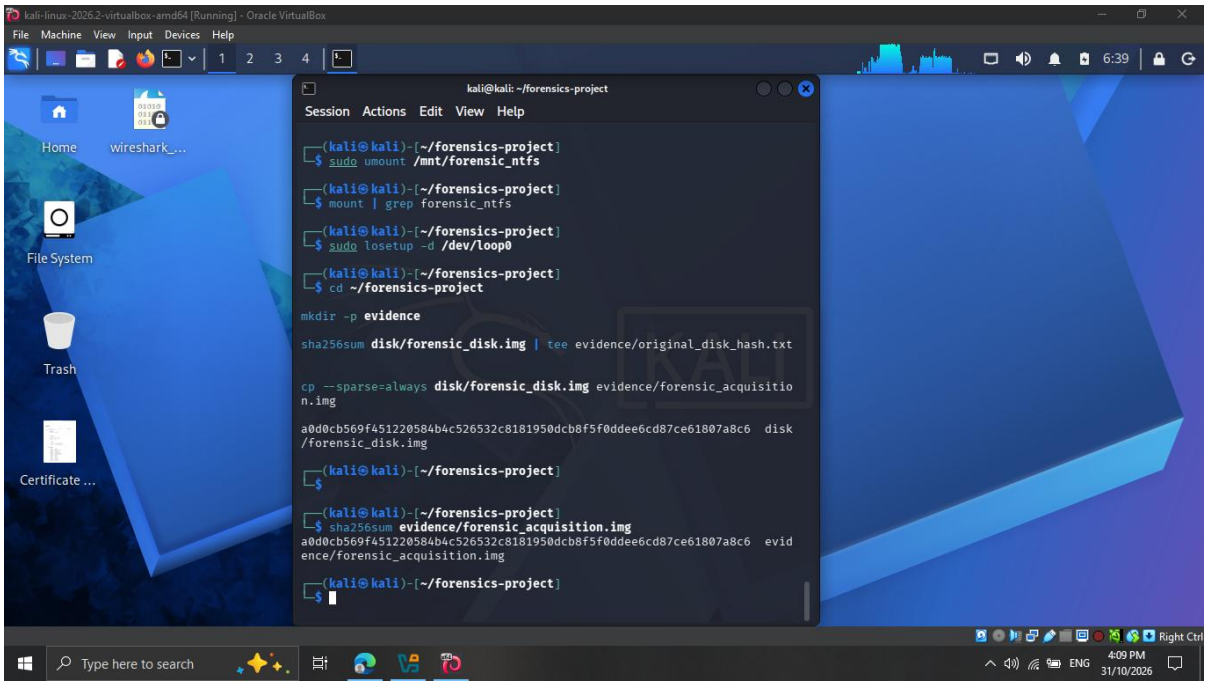
Screenshot (1281)(1).png

A14. Verifying acquisition hash, partition layout, and deleted-file listing



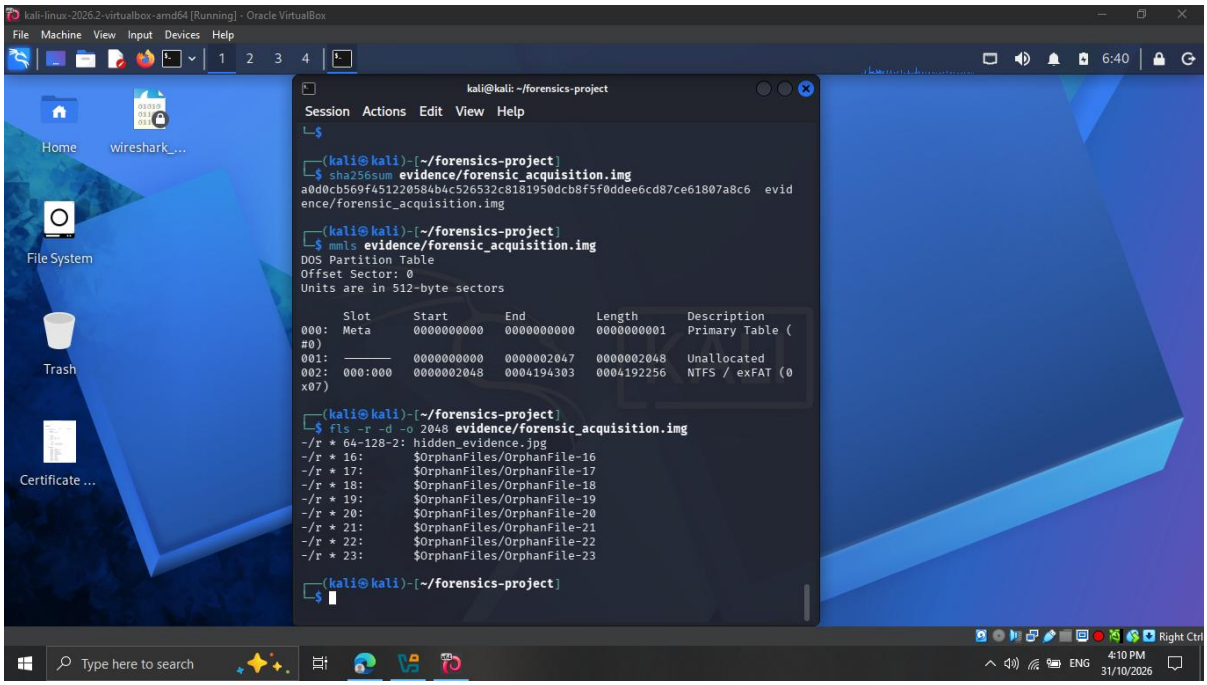
Screenshot (1282)(1).png

A15. Recovering the deleted JPG and confirming its file type and hash



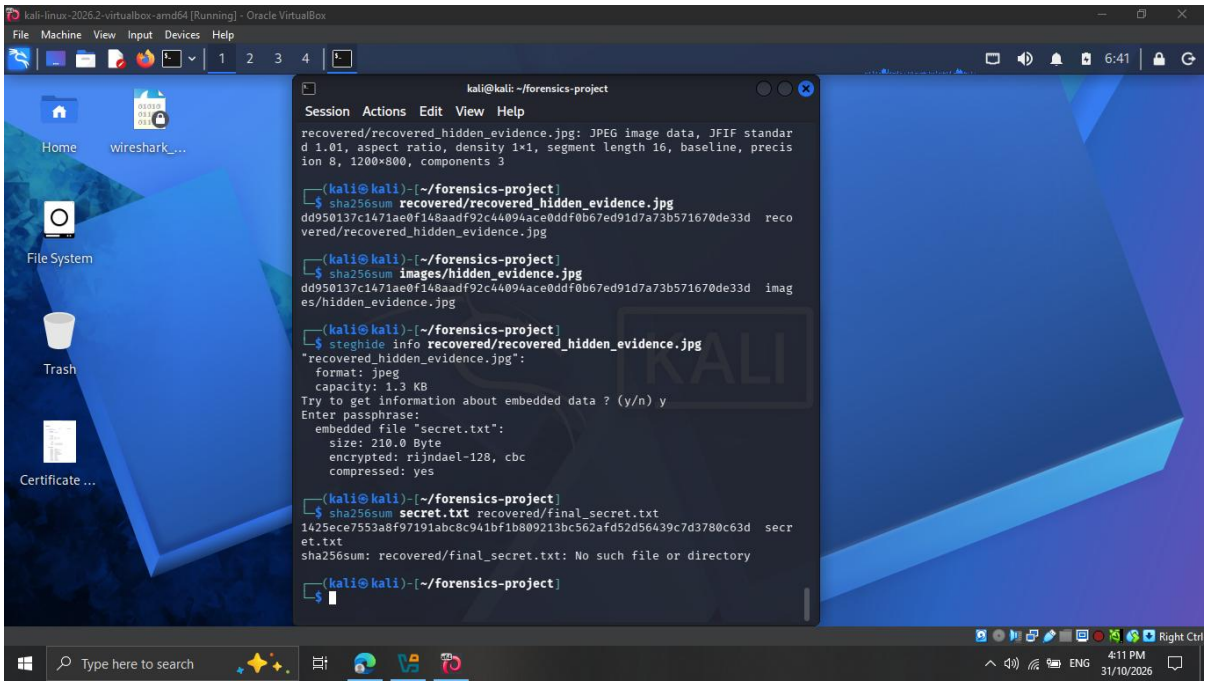
Screenshot (1283)(1).png

A16. Inspecting the recovered image for embedded Steghide data



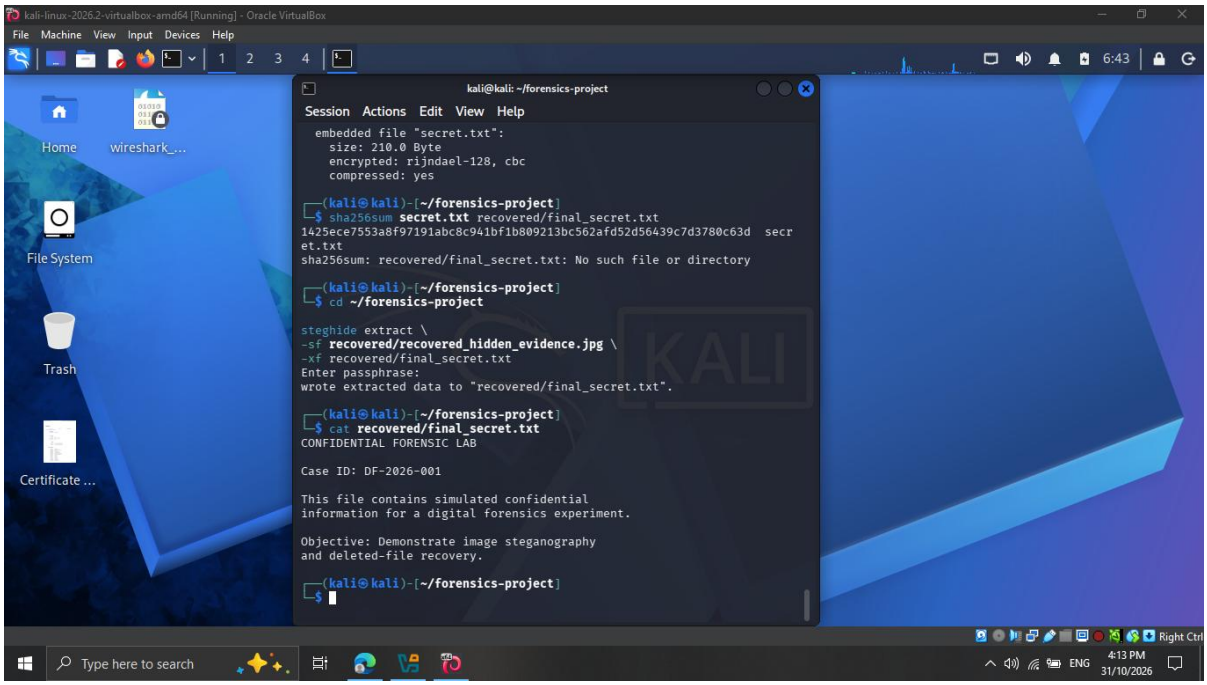
Screenshot (1284)(1).png

A17. Extracting and viewing the recovered confidential laboratory message



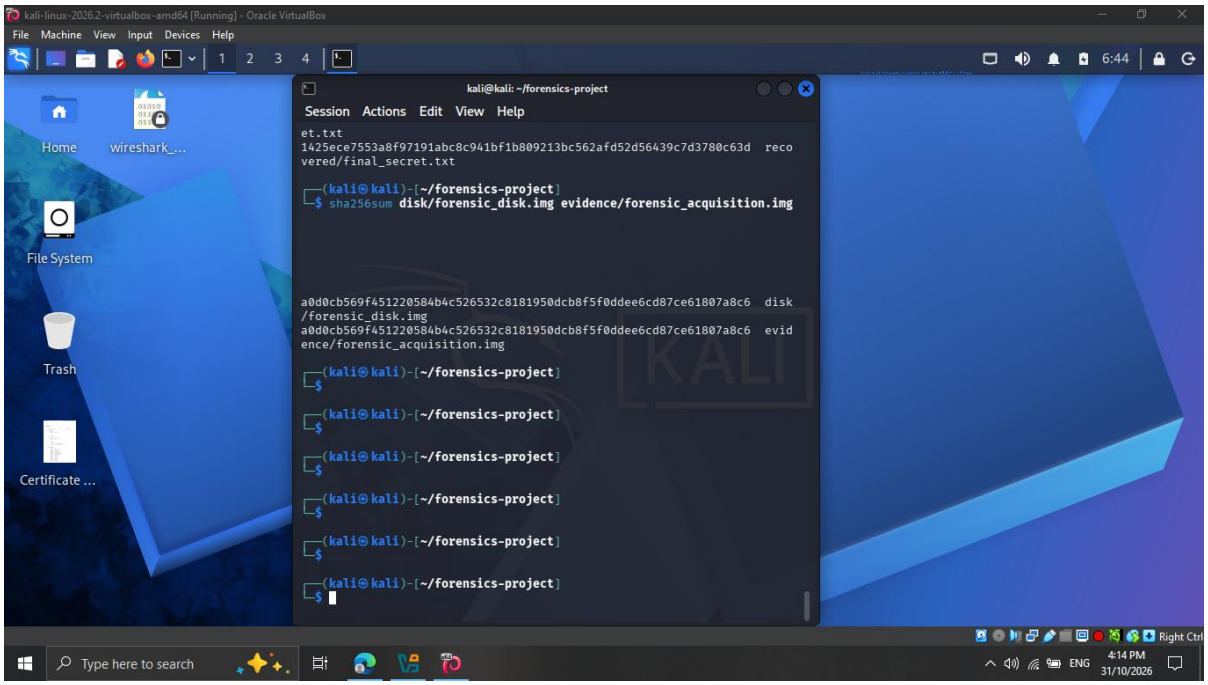
Screenshot (1285)(1).png

A18. Final SHA-256 comparison of the original and forensic acquisition images



Screenshot (1286).png

A19. Additional final terminal evidence supplied for the investigation



Screenshot (1287).png

Appendix B: Key Commands

```
# Create and verify the forensic acquisition
sha256sum disk/forensic_disk.img | tee evidence/original_disk_hash.txt
cp --sparse=always disk/forensic_disk.img evidence/forensic_acquisition.img
sha256sum evidence/forensic_acquisition.img

# Inspect partition layout
mmls evidence/forensic_acquisition.img

# List deleted entries (partition offset 2048 sectors)
fls -r -d -o 2048 evidence/forensic_acquisition.img

# Recover the deleted JPEG (metadata address 64-128-2)
icat -o 2048 evidence/forensic_acquisition.img 64-128-2 > recovered/recovered_hidden_evidence.jpg

# Verify recovered image and extract embedded text
sha256sum images/hidden_evidence.jpg recovered/recovered_hidden_evidence.jpg
steghide extract -sf recovered/recovered_hidden_evidence.jpg -xf recovered/final_secret.txt
sha256sum secret.txt recovered/final_secret.txt

# Verify original disk and acquisition copy
sha256sum disk/forensic_disk.img evidence/forensic_acquisition.img
```